

## ROS2 Day3 과제 보고서

학번: 2026406076

이름: 정창규

제출일: 2026-09-11

## 목차

### 1. Task01 – Launch + YAML을 이용한 turtlesim 제어

- 1.1 과제 개요
- 1.2 프로그램 구현
- 1.3 YAML 및 Launch 구성
- 1.4 실행 결과
- 1.5 느낀 점

### 2. Task02 – Qt 신호등 + 교차로 시뮬레이터

- 2.1 과제 개요
- 2.2 Traffic Light / Vehicle Node 구현
- 2.3 YAML 및 Launch 구성
- 2.4 Qt UI 및 Topic 연동
- 2.5 실행 결과
- 2.6 느낀 점

## 1. Task01 – Launch + YAML을 이용한 turtlesim 제어

### 1.1 과제 개요

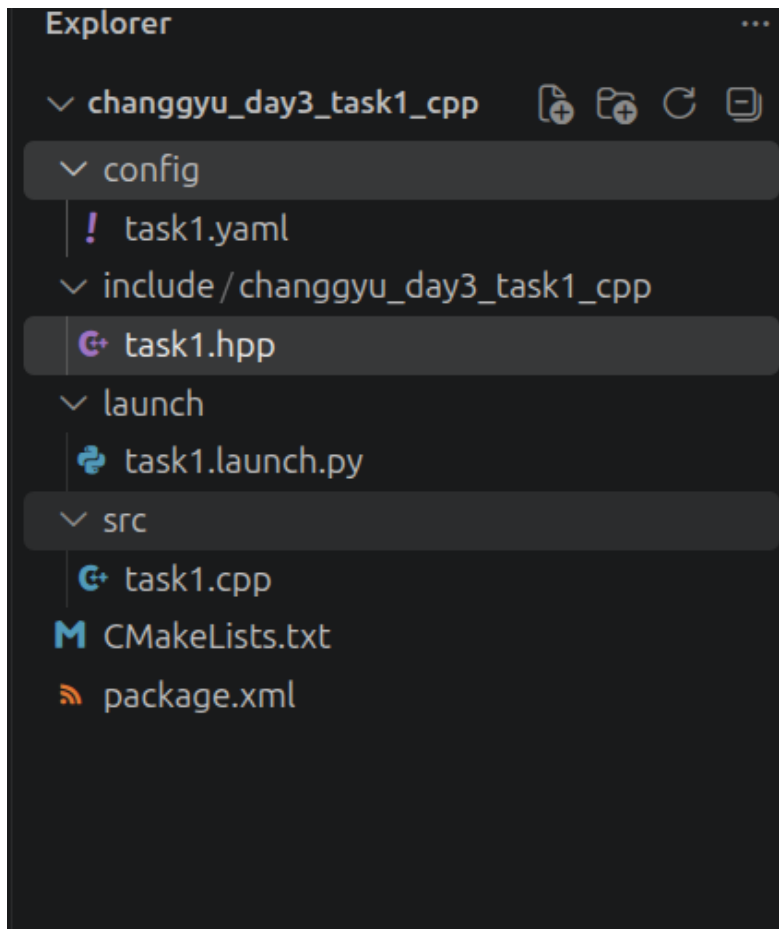
Task01에서는 ROS2 Day1의 Task03에서 작성한 turtlesim 제어 프로그램을 수정하여 선의 색상과 굵기를 Parameter로 설정하고, 해당 값을 YAML 파일에서 불러오도록 구현하였다.

기존 프로그램에서는 도형의 색상과 굵기 값을 C++ 코드 내부에서 직접 설정하였다. 이번에는 색상에 사용되는 RGB 값과 굵기 값을 Parameter로 선언하고 YAML 파일에서 관리하도록 변경하였다.

또한 Launch 파일을 작성하여 turtlesim 노드와 Task01 노드를 한 번에 실행할 수 있도록 구성하였다. 이를 통해 여러 개의 터미널에서 노드를 각각 실행하지 않고 하나의 Launch 파일을 이용하여 프로그램을 실행할 수 있도록 하였다.

### 1.2 프로그램 구현

C++ 패키지는 헤더 파일과 소스 파일을 분리하고, Parameter 값을 저장하기 위한 YAML 파일과 노드를 실행하기 위한 Launch 파일을 추가하여 다음과 같이 구성하였다.



거북이의 이동에는 기존과 동일하게 /turtle1/cmd\_vel 토픽을 사용하였으며,

geometry\_msgs/msg/Twist 메시지의 linear.x와 angular.z 값을 이용하여 직선 이동과 회전을 구현하였다.

도형을 그리는 과정에서 반복되는 직진과 회전을 하나의 polygon() 함수로 구성하였다.

```
void Task1::polygon(int sides)
{
    double angle = 2.0 * PI / sides;

    for (int i = 0; i < sides; i++)
    {
        move(1.0, 0.0, 1.5);
        move(0.0, 1.0, angle);
    }
}
```

polygon() 함수에서는 전체 회전 각도인 360도를 변의 개수로 나누어 각 도형에 필요한 회전 각도를 계산하도록 하였다.

삼각형은 polygon(3), 사각형은 polygon(4), 오각형은 polygon(5)를 호출하도록 하였다.

```
polygon(3);
polygon(4);
polygon(5);
```

삼각형은 한 변을 이동한 뒤 약 120도, 사각형은 약 90도, 오각형은 약 72도 회전하는 동작을 반복하여 그리도록 하였다.

원의 경우에는 직선 이동과 회전을 따로 수행하지 않고 선속도와 각속도를 동시에 적용하였다.

```
void Task1::circle()
{
    move(1.0, 1.0, 2.0 * PI);
}
```

거북이가 앞으로 이동하면서 동시에 계속 방향을 변경하도록 하여 원 형태의 궤적을 만들 수 있도록 하였다.

키보드 입력은 getKey() 함수를 이용하여 Enter를 누르지 않고 한 글자씩 입력받도록 하였다. 입력된 키는 run() 함수에서 판단하도록 하였으며, W, A, S, D 키를 누르면 각각 하나의 도형이 자동으로 그려지도록 구성하였다.

```
W : Triangle
A : Square
S : Pentagon
D : Circle
R : Reset
Q : Quit
```

### 1.3 YAML 및 Launch 구성

선의 색상과 굵기는 C++ 코드에 직접 값을 지정하지 않고 ROS2 Parameter로 관리하도록 수정하였다.

먼저 생성자에서 다음과 같이 Parameter를 선언하였다.

```
declare_parameter("pen_r", 255);
declare_parameter("pen_g", 0);
declare_parameter("pen_b", 0);
declare_parameter("pen_width", 3);
```

pen\_r, pen\_g, pen\_b는 RGB 색상 값을 나타내고, pen\_width는 선의 굵기를 나타낸다.

setPen() 함수에서는 get\_parameter()를 이용하여 현재 Parameter 값을 가져온 뒤 turtlesim의 set\_pen 서비스에 전달하도록 하였다.

```
req->r = get_parameter("pen_r").as_int();
req->g = get_parameter("pen_g").as_int();
req->b = get_parameter("pen_b").as_int();
req->width = get_parameter("pen_width").as_int();
```

YAML 파일에는 실제로 사용할 Parameter 값을 다음과 같이 저장하였다.

```
task1_controller:
  ros__parameters:
    pen_r: 255
    pen_g: 0
    pen_b: 0
    pen_width: 3
```

이를 통해 C++ 소스 코드를 수정하지 않고 YAML 파일의 값만 변경하여 선의 색상과 굵기를 바꿀 수 있도록 하였다.

Launch 파일에서는 task1.yaml 파일의 경로를 불러온 뒤 Task01 노드에 Parameter로 전달하도록 구성하였다.

```
Node(
  package="changgyu_day3_task1_cpp",
  executable="task1_node",
  name="task1_controller",
  parameters=[yaml_file]
)
```

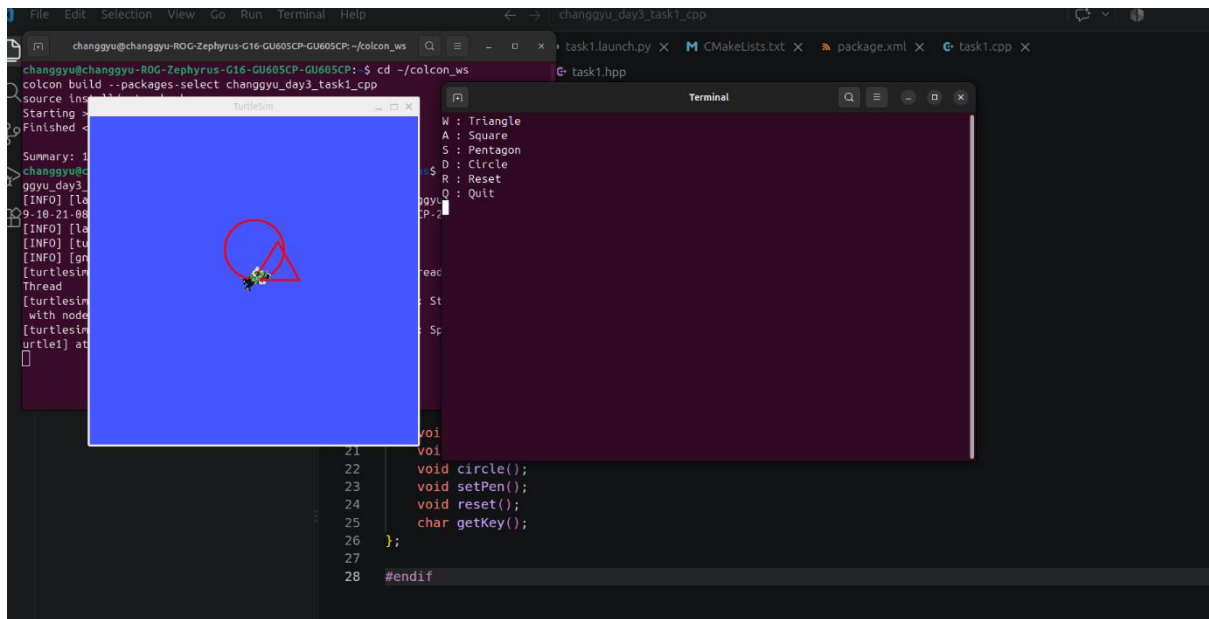
또한 turtlesim 노드와 Task01 노드를 하나의 Launch 파일에서 함께 실행하도록 하였다. Task01 노드는 키보드 입력을 사용하기 때문에 별도의 터미널에서 실행되도록 설정하였다.

## 1.4 실행 결과

프로그램 실행을 위해 패키지를 빌드한 뒤 Launch 파일을 실행하였다.

```
cd ~/colcon_ws
colcon build --packages-select changgyu_day3_task1_cpp
source install/setup.bash
ros2 launch changgyu_day3_task1_cpp task1.launch.py
```

Launch 파일을 실행하면 turtlesim 과 Task01 노드가 함께 실행되고, 별도의 터미널에서 W, A, S, D 키를 이용하여 삼각형, 사각형, 오각형, 원을 그릴 수 있었다.



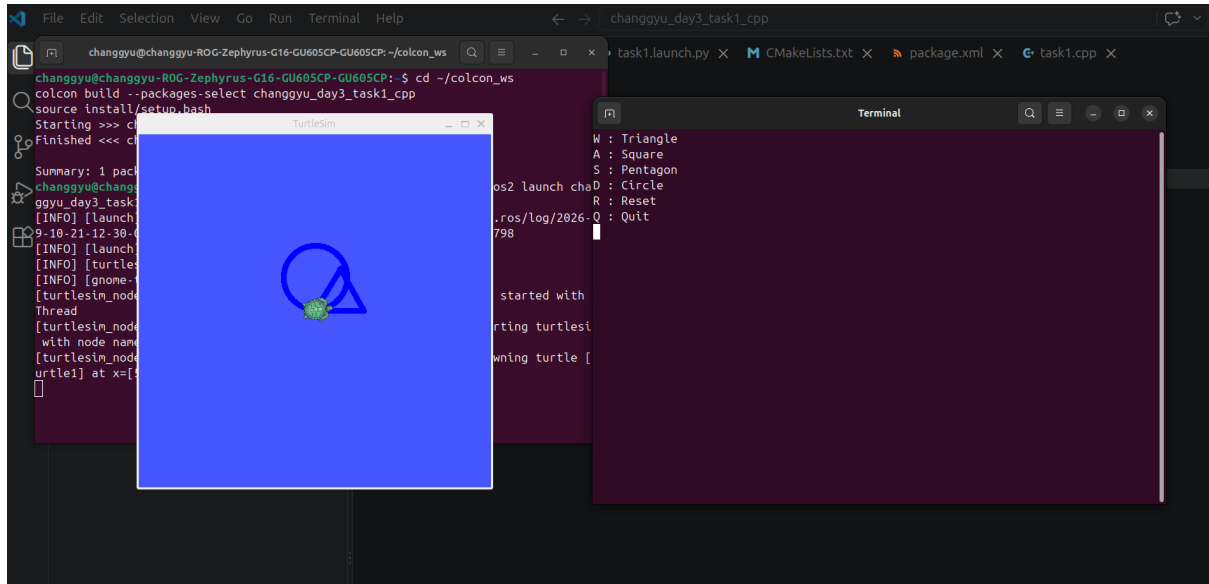
처음에는 YAML 파일을 다음과 같이 설정하였다.

```
pen_r: 255
pen_g: 0
pen_b: 0
pen_width: 3
```

빨간색과 굵기 3 이 적용된 상태에서 도형이 그려지는 것을 확인하였다.

이후 프로그램을 종료하고 YAML 파일의 값을 다음과 같이 변경하였다.

```
pen_r: 0
pen_g: 0
pen_b: 255
pen_width: 8
```



다시 프로그램을 실행한 결과 선의 색상이 빨간색에서 파란색으로 변경되었고, 선의 굵기도 이전보다 두꺼워진 것을 확인할 수 있었다.

C++ 소스 코드를 수정하거나 다시 작성하지 않고 YAML 파일의 Parameter 값만 변경하여 실행 결과를 바꿀 수 있었다.

파라미터 변경 전과 변경 후의 실제 동작 과정은 작동 영상으로 별도 첨부하였다.

## 1.5 느낀 점

Day1 Task03에서는 선의 색상과 굵기를 C++ 코드 내부에서 직접 설정하였기 때문에 값을 변경할 때마다 소스 코드를 수정해야 했다. 이번 과제에서는 Parameter와 YAML 파일을 사용하면서 프로그램에서 사용하는 설정값을 소스 코드와 분리하여 관리할 수 있다는 것을 알게 되었다.

처음에는 YAML 파일에 값을 작성하면 프로그램에서 바로 사용할 수 있다고 생각했지만, C++에서 Parameter를 먼저 선언하고 `get_parameter()`를 이용하여 값을 가져와야 한다는 것을 이해할 수 있었다.

또한 기존에는 turtlesim과 제어 프로그램을 각각 다른 터미널에서 실행하였지만, Launch 파일을 이용하여 여러 노드를 한 번에 실행할 수 있었다. 이를 통해 YAML은 프로그램에서 사용하는 설정값을 관리하고, Launch는 여러 노드와 설정을 함께 실행하는 데 사용할 수 있다는 것을 이해할 수 있었다.

## 2. Task02 – Qt 신호등 + 교차로 시뮬레이터

### 2.1 과제 개요

Task02에서는 ROS2와 Qt를 이용하여 신호등과 차량의 움직임을 확인할 수 있는 교차로 시뮬레이터를 구현하였다.

신호등의 상태를 관리하는 Traffic Light Node와 차량의 위치를 관리하는 Vehicle Node를 각각 작성하였다. 두 노드는 Topic을 이용하여 신호등의 상태를 전달하고, 차량은 현재 신호에 따라 이동하거나 정지하도록 구성하였다.

또한 신호등의 각 점등 시간과 차량의 이동 속도는 YAML 파일에서 관리하도록 하였으며, Qt에서는 ROS2 Topic을 구독하여 현재 신호등의 상태와 차량 위치를 화면에 표시하도록 구현하였다.

### 2.2 Traffic Light / Vehicle Node 구현

먼저 신호등의 상태를 관리하는 traffic\_light\_node를 작성하였다.

신호등은 RED, GREEN, YELLOW 순서로 변경되고 다시 RED로 돌아가도록 구성하였다.

```
if(state_ == "RED")
    state_ = "GREEN";
else if(state_ == "GREEN")
    state_ = "YELLOW";
else
    state_ = "RED";
```

각 신호의 유지 시간은 코드 내부에서 직접 지정하지 않고 Parameter 값을 가져와 사용하였다.

```
if(state_ == "RED")
    time = get_parameter("red_time").as_int();
else if(state_ == "GREEN")
    time = get_parameter("green_time").as_int();
else
    time = get_parameter("yellow_time").as_int();
```

현재 신호 상태는 /traffic\_light Topic을 이용하여 std\_msgs/msg/String 형태로 발행하도록 하였다.

차량의 움직임을 관리하는 vehicle\_node에서는 /traffic\_light Topic을 구독하여 현재 신호 상태를 받아오도록 하였다.

```

sub_ = create_subscription<std_msgs::msg::String>(
    "/traffic_light",
    10,
    std::bind(
        &Vehicle::lightCallback,
        this,
        std::placeholders::_1
    )
);

```

차량은 GREEN 상태에서는 정상 속도로 이동하고, YELLOW 상태에서는 이동 속도를 줄이며, RED 상태에서는 현재 위치에서 정지하도록 구성하였다.

```

if(state_ == "GREEN")
    position_ += speed * 0.1;
else if(state_ == "YELLOW")
    position_ += speed * 0.03;

```

RED 상태에서는 position\_ 값을 증가시키지 않기 때문에 차량이 정지하도록 하였다.

현재 차량의 위치는 /vehicle\_position Topic 을 이용하여 std\_msgs/msg/Float64 형태로 발행하였다.

## 2.3 YAML 및 Launch 구성

신호등의 점등 시간과 차량의 속도는 YAML 파일에서 변경할 수 있도록 Parameter 로 구성하였다.

```

traffic_light_node:
  ros__parameters:
    red_time: 5
    green_time: 5
    yellow_time: 2

vehicle_node:
  ros__parameters:
    vehicle_speed: 1.0

```

red\_time, green\_time, yellow\_time 은 각각 RED, GREEN, YELLOW 신호가 유지되는 시간을 나타내며, vehicle\_speed 는 차량의 이동 속도를 나타낸다.

이를 통해 C++ 코드 내부의 값을 직접 수정하지 않고 YAML 파일에서 신호 시간과 차량 속도를 변경할 수 있도록 하였다.

Launch 파일에서는 traffic\_light\_node, vehicle\_node, traffic\_gui 를 함께 실행하도록 구성하였다.

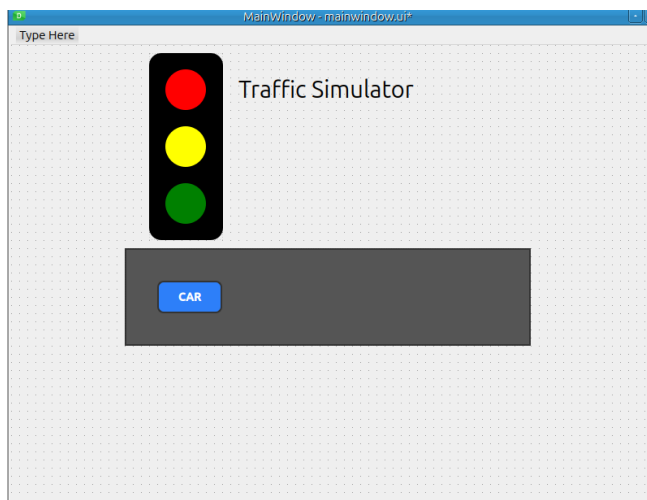
```
Node(  
  package="changgyu_day3_task2_cpp",  
  executable="traffic_light_node",  
  name="traffic_light_node",  
  parameters=[yaml_file]  
)  
  
Node(  
  package="changgyu_day3_task2_cpp",  
  executable="vehicle_node",  
  name="vehicle_node",  
  parameters=[yaml_file]  
)  
  
Node(  
  package="changgyu_day3_task2_cpp",  
  executable="traffic_gui",  
  name="traffic_gui"  
)
```

이를 통해 하나의 Launch 명령으로 신호등 노드와 차량 노드, Qt 프로그램을 동시에 실행할 수 있도록 하였다.

## 2.4 Qt UI 및 Topic 연동

Qt Designer 를 이용하여 신호등과 차량의 움직임을 확인할 수 있는 UI 를 구성하였다.

신호등은 빨간색, 노란색, 초록색을 나타내는 세 개의 Label 로 구성하였으며, 아래쪽에는 차량이 이동하는 도로와 차량 Label 을 배치하였다.



Qt 프로그램에서는 /traffic\_light 과 /vehicle\_position Topic 을 각각 구독하도록 하였다.

```
light_sub_ = node_>create_subscription<std_msgs::msg::String>(
    "/traffic_light",
    10,
    [this](std_msgs::msg::String::SharedPtr msg)
    {
        light_ = msg->data;
    }
);
```

```
vehicle_sub_ = node_>create_subscription<std_msgs::msg::Float64>(
    "/vehicle_position",
    10,
    [this](std_msgs::msg::Float64::SharedPtr msg)
    {
        position_ = msg->data;
    }
);
```

Qt 에서는 전달받은 신호 상태에 따라 현재 신호만 밝게 표시하도록 하였다.

```
ui->redLight->setStyleSheet(
    light_ == "RED"
    ? "background-color:red; border-radius:25px;"
    : "background-color:#500000; border-radius:25px;"
);
```

차량의 위치는 vehicle\_node 에서 전달받은 position\_ 값을 이용하여 화면상의 차량 Label 위치를 변경하도록 하였다.

```
int x = 10;

if(max_x > 0)
    x += static_cast<int>(
        std::fmod(position_ * 40.0, max_x)
    );

ui->carLabel->move(x, ui->carLabel->y());
```

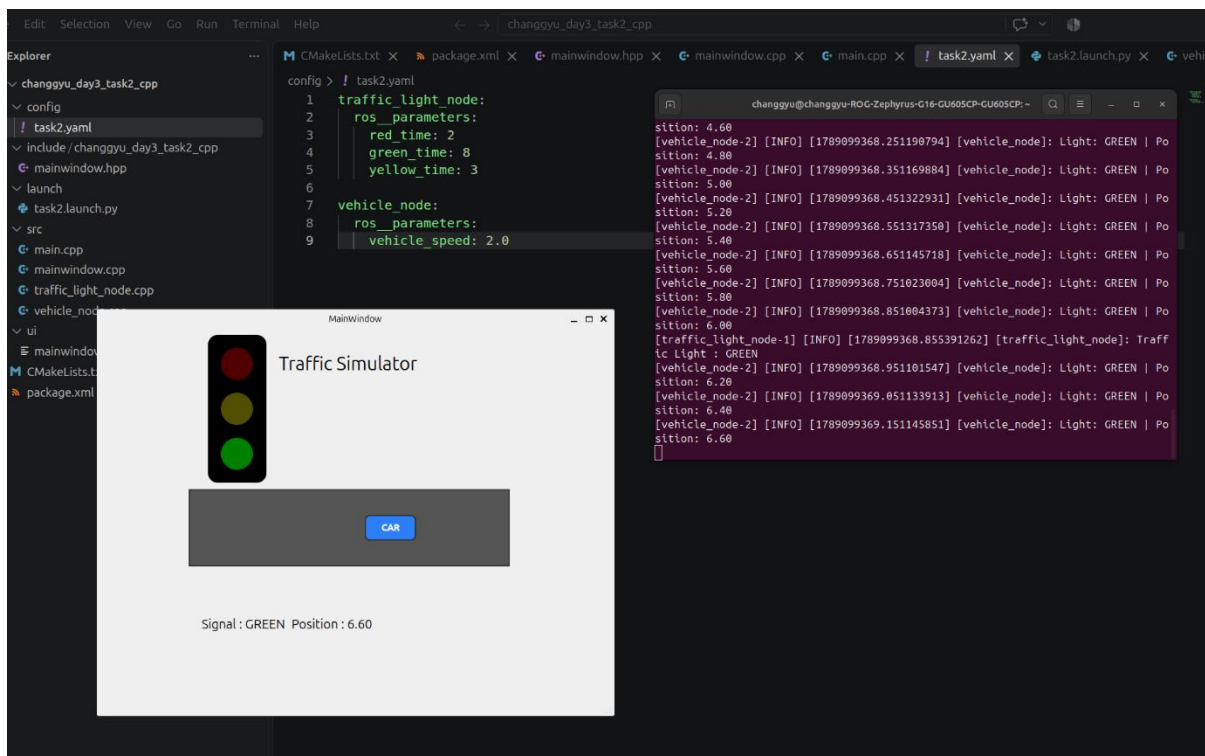
Qt 에서는 차량의 이동 여부를 직접 판단하지 않고, ROS2 Topic 을 통해 전달받은 신호 상태와 차량 위치를 화면에 표시하도록 구성하였다.

## 2.5 실행 결과

프로그램을 실행하기 위해 패키지를 빌드한 뒤 Launch 파일을 실행하였다.

```
cd ~/colcon_ws
colcon build --packages-select changgyu_day3_task2_cpp
source install/setup.bash
ros2 launch changgyu_day3_task2_cpp task2.launch.py
```

Launch 파일을 실행하면 신호등 노드와 차량 노드가 실행되고 Qt 시뮬레이터 창이 함께 나타났다.



위 실행 화면에서는 변경된 Parameter 값이 적용된 상태에서 Qt 시뮬레이터와 터미널 출력이 정상적으로 동작하는 것을 확인하였다.

처음에는 신호 시간을 RED 5 초, GREEN 5 초, YELLOW 2 초로 설정하고 차량 속도를 1.0 으로 설정하였다.

```
red_time: 5
green_time: 5
yellow_time: 2

vehicle_speed: 1.0
```

실행 결과 RED 상태에서는 차량이 정지하고, GREEN 상태에서는 차량이 이동하였으며, YELLOW 상태에서는 이동 속도가 감소하는 것을 확인하였다.

이후 YAML 파일의 값을 다음과 같이 변경하였다.

```
red_time: 2
green_time: 8
yellow_time: 3

vehicle_speed: 2.0
```

다시 실행한 결과 RED 신호의 유지 시간이 줄어들고 GREEN 신호의 유지 시간이 증가하였다. 또한 vehicle\_speed 값을 증가시키면서 차량이 이전보다 빠르게 이동하는 것을 확인하였다.

Parameter 변경 전과 변경 후의 실제 동작 과정은 작동 영상으로 별도 첨부하였다.

## 2.6 느낀 점

이번 과제를 진행하면서 ROS2 에서 여러 노드가 Topic 을 통해 데이터를 주고받고, 그 데이터를 Qt 에서 시각적으로 표현하는 과정을 직접 구현할 수 있었다.

처음에는 신호등과 차량의 움직임을 모두 Qt 프로그램 내부에서 처리하는 방식으로 생각했지만, 신호등의 상태 변경은 traffic\_light\_node, 차량의 이동은 vehicle\_node 에서 처리하고 Qt 에서는 전달받은 값을 화면에 표시하도록 역할을 나누어 구현하였다.

또한 YAML 파일을 이용하여 신호등의 점등 시간과 차량 속도를 변경해보면서 프로그램의 설정값을 소스 코드와 분리하여 관리할 수 있다는 것을 다시 확인하였다.

Launch 파일을 통해 세 개의 프로그램을 한 번에 실행해보면서 여러 노드가 함께 동작하는 ROS2 프로그램의 구조를 이전보다 더 이해할 수 있었다.